Code Explanation

Testing Delta Live Tables

The provided code is a test suite for Delta Live Tables (DLT) pipelines using Pytest and PySpark. It tests two functions: `order_silver` and `customeraggregatespend`. The tests mock the `dlt` module, which is only available in Databricks runtime, to test the DLT pipelines.

The test suite creates a Spark session for testing and uses the `@patch` decorator to mock the `dlt` module. The tests verify the correctness of the `order_silver` and `customeraggregatespend` functions by checking the output DataFrames.

Overview of the Code

The code is a Pytest test suite that tests two DLT pipeline functions. It creates a Spark session, mocks the `dlt` module, and tests the `order_silver` and `customeraggregatespend` functions.

Step 1: Creating a Spark Session for Testing
This step creates a Spark session with the necessary configurations for testing DLT pipelines.

```
@pytest.fixture(scope="module")
def spark():
    """Create a Spark session for testing."""
    return SparkSession.builder
        .appName("Test DLT Pipeline")
        .master("local[*]")
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSess
ionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark
.sql.delta.catalog.DeltaCatalog")
        .getOrCreate()
```

Step 2: Testing the `order_silver` Function
This step tests the `order_silver` function by creating mock bronze data, mocking the `dlt.read` method, and verifying the output DataFrame.

```
@patch('src.delta_live_tables.dlt')
def test_order_silver(mock_dlt, spark):
    """Test the order_silver function."""
    # Create mock bronze data
    bronze_data = [
        (101, "Item1", 10.0, 2, "2023-01-01", 1),
        (101, "Item1", 10.0, 2, "2023-01-01", 1),  # Duplicate
        (102, "Item2", None, 3, "2023-01-02", 2),  # Null price
        (103, "Item3", 15.0, 1, "2023-01-03", 3)
    ]
    bronze_schema = ["OrderId", "ItemName", "PricePerUnit", "Qty", "
Date", "CustId"]
    bronze_df = spark.createDataFrame(bronze_data, bronze_schema)

    # Mock dlt.read to return our test data
    mock_dlt.read.return_value = bronze_df
```

```
    # Import the function after mocking
    from src.delta_live_tables import order_silver

    # Call the function
    result_df = order_silver()

    # Verify results
    assert result_df.count() == 2  # Should remove duplicates and nu
lls
```

Step 3: Verifying the `order_silver` Function Output
This step verifies the output of the `order_silver` function by checking the `TotalAmount` calculation.

```
# Verify TotalAmount calculation
total_amount = result_df.filter(F.col("OrderId") == 101).select("Tot
alAmount").collect()[0][0]
assert total_amount == 20.0  # 10.0 * 2 = 20.0

# Verify TotalAmount calculation for another record
total_amount = result_df.filter(F.col("OrderId") == 103).select("Tot
alAmount").collect()[0][0]
assert total_amount == 15.0  # 15.0 * 1 = 15.0
```

Step 4: Testing the `customeraggregatespend` Function
This step tests the `customeraggregatespend` function by creating mock `ordersummary` and `order_silver` data, mocking the `dlt.read` method, and verifying the output DataFrame.

```
@patch('src.delta_live_tables.dlt')
def test_customeraggregatespend(mock_dlt, spark):
    """Test the customeraggregatespend function."""
    # Create mock ordersummary data
    ordersummary_data = [
        (1, "John Doe", "john@example.com", "North", 101, "Item1", 1
0.0, 2, "2023-01-01", "2023-01-01 00:00:00", None, True),
        (1, "John Doe", "john@example.com", "North", 102, "Item2", 2
0.0, 1, "2023-01-01", "2023-01-01 00:00:00", None, True),
        (2, "Jane Smith", "jane@example.com", "South", 103, "Item3",
 15.0, 3, "2023-01-02", "2023-01-01 00:00:00", None, True)
    ]
    ordersummary_schema = ["CustId", "Name", "EmailId", "Region", "O
rderId", "ItemName", "PricePerUnit", "Qty", "Date", "StartDate", "En
dDate", "IsActive"]
    ordersummary_df = spark.createDataFrame(ordersummary_data, order
summary_schema)

    # Create mock order_silver data
    order_data = [
        (101, "Item1", 10.0, 2, "2023-01-01", 1, 20.0),
        (102, "Item2", 20.0, 1, "2023-01-01", 1, 20.0),
        (103, "Item3", 15.0, 3, "2023-01-02", 2, 45.0)
    ]
    order_schema = ["OrderId", "ItemName", "PricePerUnit", "Qty", "D
ate", "CustId", "TotalAmount"]
```

```
    order_df = spark.createDataFrame(order_data, order_schema)

    # Mock dlt.read to return our test data
    mock_dlt.read.side_effect = lambda table_name: ordersummary_df i
f table_name == "ordersummary" else order_df

    # Import the function after mocking
    from src.delta_live_tables import customeraggregatespend

    # Call the function
    result_df = customeraggregatespend()
```

Step 5: Verifying the `customeraggregatespend` Function Output

This step verifies the output of the `customeraggregatespend` function by checking the aggregation results.

```
# Verify results
assert result_df.count() == 2  # Two distinct Name-Date combinations

# Check aggregation for John Doe
john_total = result_df.filter(F.col("Name") == "John Doe").select("T
otalAmount").collect()[0][0]
assert john_total == 40.0  # 20.0 + 20.0 = 40.0

# Check aggregation for Jane Smith
jane_total = result_df.filter(F.col("Name") == "Jane Smith").select(
"TotalAmount").collect()[0][0]
assert jane_total == 45.0
```